Moderated Learning Management System

Software Engineering Final Project Report

Course: CS4433 — Software Engineering

Project Type: Individual Final Project

Submission Date: 7 November 2026

Table of Contents

1. Introduction
2. Project Management & Risk
3. Requirements Engineering
   3.1. Stakeholder Analysis
   3.2. Software Requirements Specification (SRS)
   3.3. Use Case Model
4. System Design
   4.1. Architecture
   4.2. Class Diagram & Design Patterns
   4.3. Behavioural Models
   4.4. Database Design
5. Implementation
6. Verification and Validation
7. Configuration Management & QA
8. Deployment
9. Maintenance
10. Professional, Legal and Ethical Issues
11. Conclusion
12. References

1. Introduction

This report documents the complete software engineering lifecycle for a Moderated Learning Management System (LMS), developed for a secondary school. The system manages the end-to-end assessment process: Lecturers create assignments and mark submitted work; Heads of Department (HoDs) moderate and release marks; and Students submit work and view their final grades once released.

The project followed the Waterfall Model, supported by a V-Model verification strategy. The application is a C++17 console application with persistent SQLite storage. It is designed to operate offline on a single school computer, accommodating the school's unreliable internet connectivity and older hardware.

2. Project Management and Risk

Before development began, a feasibility study confirmed that the project was technically and economically viable. A Work Breakdown Structure (WBS) mapped the seven-week schedule, while critical path analysis identified Implementation and Verification as the highest-risk phases.

A Risk Register was maintained throughout the project and identified ten risks. Two particularly important mitigation strategies were:

- R2 — C++ memory bugs: the use of Resource Acquisition Is Initialization (RAII), smart pointers, and static analysis to reduce memory leaks and undefined behaviour.
- R3 — Schedule overrun: prioritisation of "Must" requirements from the MoSCoW-prioritised requirements set, ensuring that essential functionality was completed first.

Figure: Risk Register — risk-register.png (insert the exported Risk Register table/image in the diagrams folder)

3. Requirements Engineering

3.1 Stakeholder Analysis

Initial interviews with stakeholders—including the Principal, Lecturers, HoDs, Students, and IT staff—revealed several conflicting requirements. The most critical conflict concerned mark visibility: the Principal required that students must not see marks until moderation was complete, whereas Lecturers wanted to release marks immediately after marking.

This conflict was resolved by implementing a strict state-machine moderation workflow. Marks remain hidden from Students until the HoD explicitly releases them. A second conflict concerned late penalties; this was resolved by adopting the HoD's policy of a 10% penalty per day, up to a maximum of three days, while ensuring that late work is still accepted and marked rather than rejected outright.

3.2 Software Requirements Specification (SRS)

The system specification contains 15 functional requirements and 10 non-functional requirements. Requirements were prioritised using the MoSCoW method.

Key Functional Requirements

- FR10: The system shall calculate a late penalty of 10% of the available mark per day, for up to three days. After three days, the mark shall be zero.
- FR14: The system shall prevent Students from viewing marks until the marks have been released by the HoD.
- FR15: The system shall record every mark change in an audit log.

Key Non-Functional Requirements

- NFR1: Login shall complete within two seconds with 800 users stored in the database.
- NFR3: Passwords must be stored as salted hashes.

3.3 Use Case Model

The Use Case Diagram models the interactions between the four system actors: Student, Lecturer, HoD, and Admin. Three fully dressed use cases were specified:

- Submit Assignment, including late-submission and resubmission paths.
- Moderate and Release Marks.
- Grant Extension.

Figure: Use Case Diagram — use-case-diagram.png (shows actor interactions and system boundary)

4. System Design

4.1 Architecture

The system uses a Three-Tier Layered Architecture consisting of a Presentation Layer, Business Logic Layer, and Data Access Layer. This architecture was selected because it provides clear separation of concerns, improves testability, and supports secure access to persistent data.

A Model-View-Controller (MVC) architecture was considered but rejected. Because the system is a simple console application without multiple simultaneous views, MVC would introduce unnecessary complexity.

Figure: Component Diagram — component-diagram.png (shows the three-tier architecture and component dependencies)

4.2 Class Diagram and Design Patterns

The design-level Class Diagram shows an inheritance hierarchy in which an abstract User class is extended by Student, Lecturer, HoD, and Admin classes. Four key design patterns were applied:

- Strategy Pattern: encapsulates late-penalty rules, allowing the penalty algorithm to be changed without modifying the Submission class.
- Repository Pattern: isolates database access from business logic.
- Factory Pattern: handles creation of different User roles.
- Observer Pattern: notifies Students when marks are released.

Figure: Class Diagram — class-diagram.png (shows inheritance, associations, and design-pattern structure)

4.3 Behavioural Models

A State Machine Diagram models the assessment lifecycle. A submission moves through the following states:

SUBMITTED → MARKED → MODERATED → RELEASED

A Sequence Diagram details the “Moderate and Release Marks” use case. It shows how the HoD triggers updates across the database, submission status, and audit log.

Figure: State Machine Diagram — state-machine-diagram.png (models submission lifecycle states)

Figure: Sequence Diagram — sequence-diagram.png (models the Moderate and Release Marks workflow)

4.4 Database Design

The database was normalised to Third Normal Form (3NF). The Entity Relationship Diagram (ERD) shows the relationships between Users, Submissions, and Marks.

Single-table inheritance is used for the Users table to avoid unnecessary joins. A role discriminator column identifies whether a record represents a Student, Lecturer, HoD, or Admin. All queries in the Data Access Layer use parameterised queries to prevent SQL injection.

Figure: Entity Relationship Diagram — erd.png (shows entities, attributes, and relationships)

5. Implementation

The system was implemented in C++17 and is hosted in a GitHub repository. The main implementation features are described below.

- Security: Passwords are never stored in plain text. A salted hashing algorithm is applied before database insertion.
- Persistence: SQLite handles all persistent storage and is accessed strictly through the Data Access Layer.
- Audit Logging: Every mark change triggers an INSERT into the AuditLog table, recording the old value, new value, and user ID.
- Code Quality: The code follows strict naming conventions, and memory is managed using smart pointers and RAII to prevent memory leaks.

6. Verification and Validation

Testing followed the V-Model and was mapped directly to the corresponding design phases. Test cases were designed using Equivalence Partitioning and Boundary Value Analysis. For example, the late-penalty functionality was tested using 0, 1, 3, and 4 days late.

Automated tests were implemented using GoogleTest. The continuous integration pipeline runs these tests on every push to the repository. The Business Logic Layer achieved 78% line coverage.

Testing revealed a critical defect in which a Student could view a mark immediately after the Lecturer marked it. The defect was fixed by updating the SQL query so that marks are returned only when the submission status is RELEASED.

Figure: CI Pipeline Screenshot — ci-pipeline.png (shows a successful GitHub Actions run)

7. Configuration Management and Quality Assurance

- Git Branching: A Feature Branch workflow was used. Code is merged into main only through Pull Requests after passing continuous integration.
- CI Pipeline: GitHub Actions builds the project and runs automated tests on every push.
- Static Analysis: cppcheck was used to identify uninitialized variables and potential memory leaks. All reported findings were resolved.
- Metrics: The cyclomatic complexity of Business Logic functions was kept below 10 to support maintainability.

8. Deployment

The deployment package includes the compiled lms_app.exe executable, the sqlite3.dll library, and the lms.db database file. A UML Deployment Diagram illustrates how these components are mapped to the school's hardware.

Because the system is a command-line application, user manuals were created for the Admin, Lecturer, HoD, and Student roles. Each manual explains the terminal commands required for that role.

Figure: Deployment Diagram — deployment-diagram.png (maps software artifacts to school hardware)

9. Maintenance

Following the release of version 1.0, Change Request CR1 was submitted to allow class-wide deadline extensions. The request was classified as a Functional Enhancement.

- Impact: The change required updates to the Extension database table and the StudentMenu command-line interface.
- Implementation: A grantClassExtension() method was added to the Business Logic Layer.
- Regression Testing: The CI suite was rerun, and new tests were added to ensure that individual extensions and late penalties continued to work correctly.

Version 1.1 was deployed successfully after regression testing.

10. Professional, Legal and Ethical Issues

- Data Protection: The system stores sensitive learner data, including names and marks. Compliance with data protection legislation, such as POPIA and GDPR, is supported through data minimisation and secure password hashing.
- Fairness and Ethics: The late-penalty rule of 10% per day is applied uniformly. However, Lecturers can grant extensions, ensuring that students with legitimate difficulties are treated fairly.
- Accessibility: As a command-line interface, the system lacks modern accessibility features such as screen-reader compatibility. This is a known limitation, mitigated through user manuals and staff training.
- Security of Records: The audit log ensures that all mark changes are traceable, helping to prevent unauthorised grade manipulation.

11. Conclusion

The Moderated Learning Management System successfully fulfils all “Must” requirements defined in the SRS. Through a strict layered architecture, a normalised database, secure authentication, and rigorous V-Model testing, the final product is secure, reliable, and maintainable.

The system addresses the school’s operational constraints—offline operation and simplicity—while enforcing the formal moderation process required by the Head of Department.

12. References

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- CS4433 Final Project Brief (2026).

Appendix A: Diagram File Structure

Place all diagram images in a folder named diagrams in the root of your GitHub repository. Use the following exact file names so that the hyperlinks in this document work after uploading:

- diagrams/risk-register.png
- diagrams/use-case-diagram.png
- diagrams/component-diagram.png
- diagrams/class-diagram.png
- diagrams/state-machine-diagram.png
- diagrams/sequence-diagram.png
- diagrams/erd.png
- diagrams/ci-pipeline.png
- diagrams/deployment-diagram.png